



Introduction to Cognitive Science

HomeWork #2



Sina Pirmoradian : 810101125

HW #2 Part #3

Connect Google Drive

```
1 from google.colab import drive
2 drive.mount('/content/drive')

Mounted at /content/drive
```

Import Required Library

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
```

Load the Data

```
1 # Load the time series data and labels from numpy files
2 data = np.load('/content/drive/MyDrive/Cognitive/HW_2/assignment2-face-data.npy')
3 labels = np.load('/content/drive/MyDrive/Cognitive/HW_2/assignment2-face-data-labels.npy')
4 print(labels.shape)

(77, 1)
```

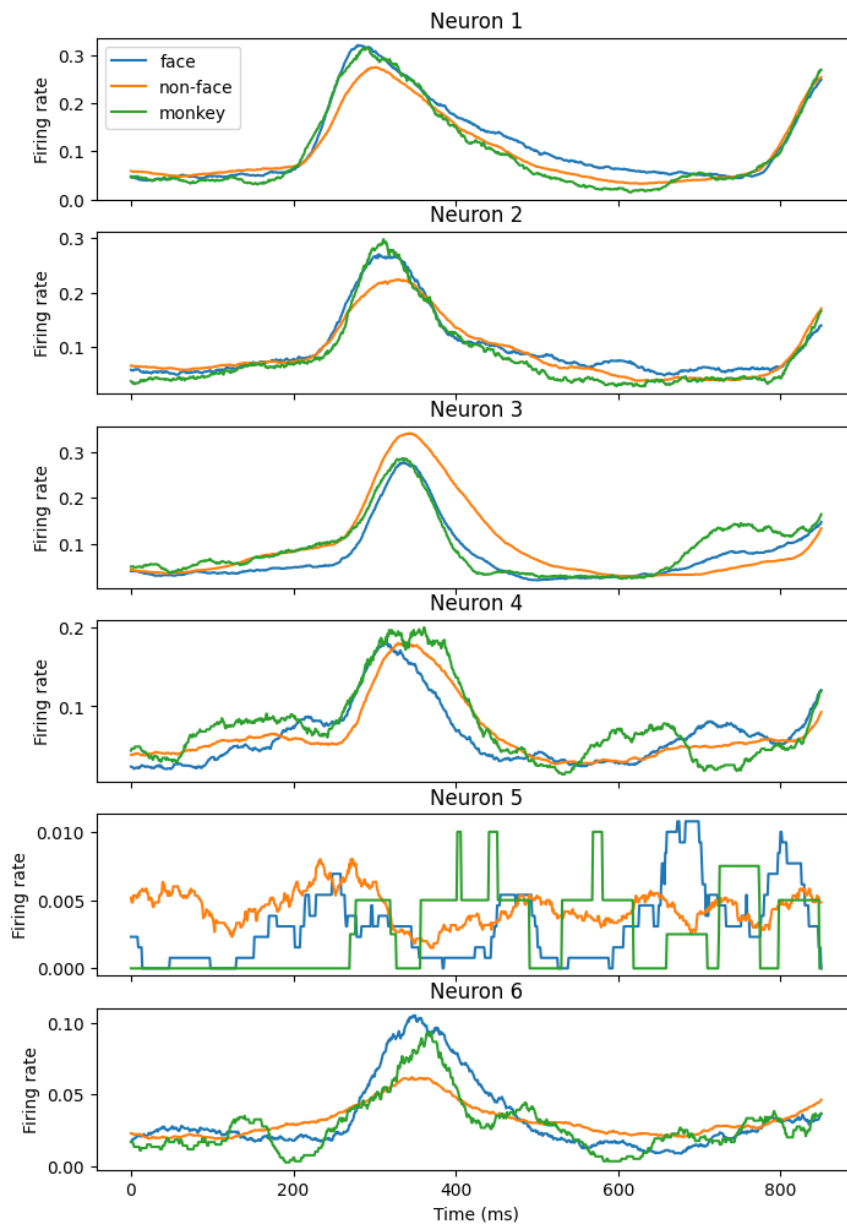
Moving Average through the Time

```
1 # Calculate firing rate using a moving average window of length 50
2 window_length = 50
3 str_ = 1
4
5 Moving_average = np.zeros((data.shape[0], data.shape[1], (data.shape[2] - window_length + 1)))
6 for i in range(data.shape[0]):
7     for j in range(data.shape[1]):
8         Moving_average[i,j] = np.convolve(data[i,j,:], np.ones(window_length)/window_length, mode='valid')[::str_]
```

Visualization Moving Average through the Time

```
1 # Plot the time series of each neuron's response to face and non-face stimuli
2 fig, axs = plt.subplots(nrows=6, ncols=1, figsize=(8,12), sharex=True)
3 for i in range(6):
4     face_trials = np.where(labels == 0)[0]
5     nonface_trials = np.where(labels == 2)[0]
6     monkey_trials = np.where(labels == 1)[0]
7     axs[i].plot(np.mean(Moving_average[face_trials,i,:], axis=0), label='face')
8     axs[i].plot(np.mean(Moving_average[nonface_trials,i,:], axis=0), label='non-face')
9     axs[i].plot(np.mean(Moving_average[monkey_trials,i,:], axis=0), label='monkey')
10    axs[i].set_ylabel('Firing rate')
11    axs[i].set_title('Neuron {}'.format(i+1))
12 axs[0].legend()
13 plt.xlabel('Time (ms)')
14 plt.show()
```





Calculate and Visualizing Firing Rate

```

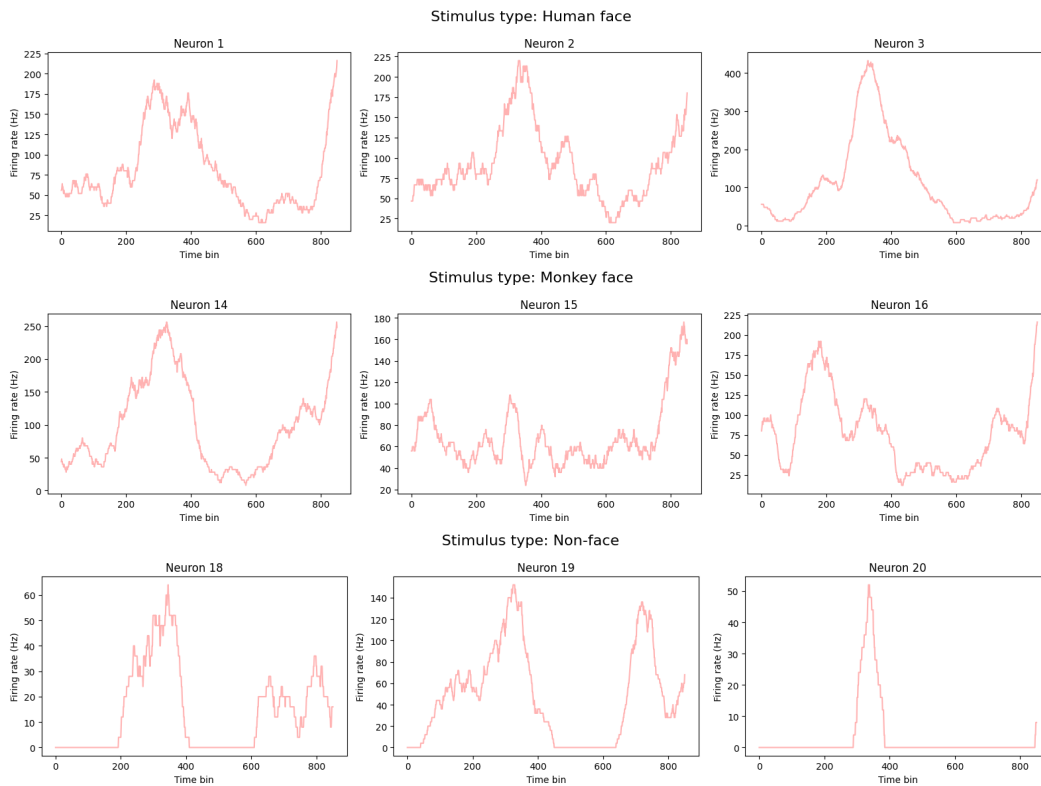
1 # Define the length and stride of the sliding window
2 window_len = 50
3 stride = 1
4
5 # Calculate the number of time bins
6 num_bins = int((data.shape[-1] - window_len) / stride) + 1
7
8 # Initialize an empty array to store the firing rates
9 firing_rates = np.zeros((len(labels), data.shape[1], num_bins))
10
11 # Loop over each trial and each neuron, and calculate the firing rates for each time bin
12 for i in range(len(labels)):
13     for j in range(data.shape[1]):
14         for k in range(num_bins):
15             window_start = k * stride
16             window_end = window_start + window_len
17             num_spikes = np.sum(data[i, j, window_start:window_end])
18             firing_rates[i, j, k] = num_spikes / window_len * 1000 # Convert to firing rate in Hz
19
20 # Plot the firing rates for each neuron, stratified by stimulus type
21 stimulus_types = ['Human face', 'Monkey face', 'Non-face']
22 num_neurons_per_stimulus = 3 # Modify this to change the number of neurons per stimulus to plot
23 for stimulus in range(3):
24     num_neurons = np.sum(labels == stimulus)
25     neuron_idx = np.where(labels == stimulus)[0][:num_neurons_per_stimulus] # Select the first `num_neurons_per_stimulus` neurons
26     num_cols = 3 # Set the number of columns to 3

```

```

27 num_rows = 1 # Set the number of rows to 1
28 fig, axs = plt.subplots(nrows=num_rows, ncols=num_cols, figsize=(16, 4))
29 fig.suptitle(f'Stimulus type: {stimulus_types[stimulus]}', fontsize=16)
30 for n, neuron in enumerate(neuron_idx):
31     axs[n].plot(firing_rates[i, neuron, :].T, color='red', alpha=0.3)
32     axs[n].set_title(f'Neuron {neuron+1}')
33     axs[n].set_xlabel('Time bin')
34     axs[n].set_ylabel('Firing rate (Hz)')
35 plt.tight_layout()
36 plt.show()
37

```



Firing rate, Neuron activity for 3 class

```

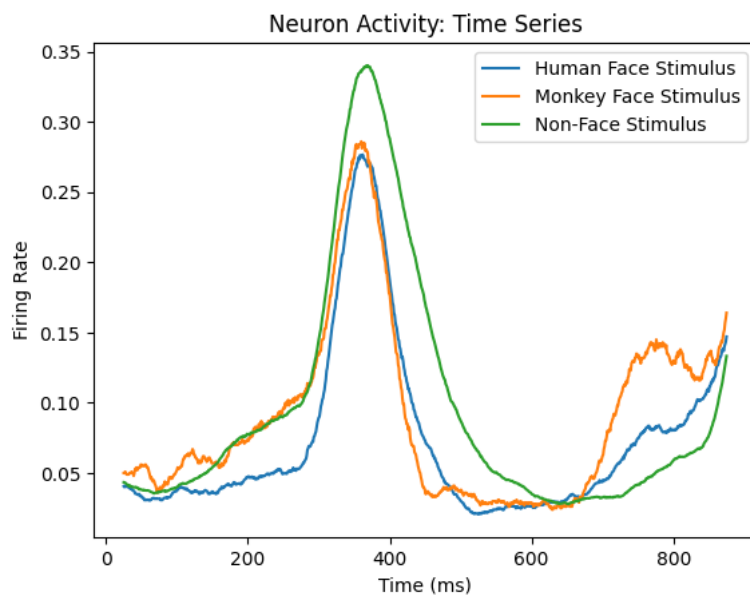
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Select a single neuron (neuron_index) and extract the spiking activity data
5 neuron_index = 2
6 neuron_data = data[:, neuron_index, :]
7
8 # Ensure that the labels array has the same shape as the number of observations/stimuli (77)
9 labels = np.load('/content/drive/MyDrive/Cognitive/HW_2/assignment2-face-data-labels.npy')[:77]
10 labels = labels.ravel()

```

```

11
12 # Split the data based on the stimulus labels
13 human_face_data = neuron_data[labels == 0]
14 monkey_face_data = neuron_data[labels == 1]
15 non_face_data = neuron_data[labels == 2]
16
17 # Calculate the firing rate by moving average window
18 window_length = 50
19 stride = 1
20
21 def calculate_firing_rate(data):
22     num_trials, num_samples = data.shape
23     firing_rate = np.zeros(num_samples - window_length + 1)
24     for i in range(num_samples - window_length + 1):
25         firing_rate[i] = np.mean(data[:, i:i+window_length])
26     return firing_rate
27
28 human_face_firing_rate = calculate_firing_rate(human_face_data)
29 monkey_face_firing_rate = calculate_firing_rate(monkey_face_data)
30 non_face_firing_rate = calculate_firing_rate(non_face_data)
31
32 # Plot the time series of firing rate for each type of stimulus
33 time = np.arange(window_length//2, 900 - window_length//2 + 1) # Adjust x-axis based on window length
34 plt.plot(time, human_face_firing_rate, label='Human Face Stimulus')
35 plt.plot(time, monkey_face_firing_rate, label='Monkey Face Stimulus')
36 plt.plot(time, non_face_firing_rate, label='Non-Face Stimulus')
37 plt.xlabel('Time (ms)')
38 plt.ylabel('Firing Rate')
39 plt.title('Neuron Activity: Time Series')
40 plt.legend()
41 plt.show()
42

```



Calculating Mutual Information

```

1 from sklearn.feature_selection import mutual_info_classif as MIC
2 labels=labels.ravel()
3 mi_score = []
4 for i in range(40):
5     mi_score.append(MIC(Moving_average[:, i, :], labels))

1 mi_score = np.array(mi_score)
2 print(mi_score.shape)

(40, 851)

1 means = np.array(mi_score).mean(axis=0)
2 stds = np.array(mi_score).std(axis=0)
3
4 means = pd.DataFrame(means).rolling(50, axis = 0).mean().dropna(axis=0).to_numpy().reshape(-1)
5 stds = pd.DataFrame(stds).rolling(50, axis = 0).mean().dropna(axis=0).to_numpy().reshape(-1)

1 print(means.shape)
2 print(stds.shape)

```

```

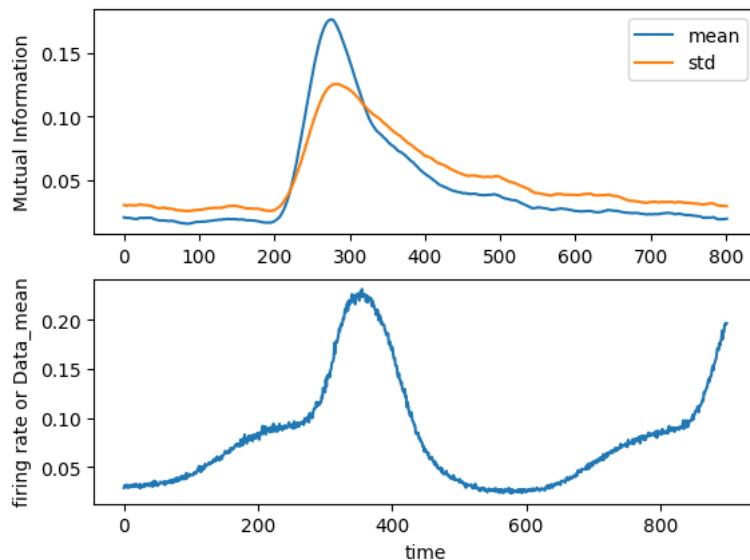
4 print(status.shape)

(802,)
(802,)

1 plt.subplot(2, 1, 1)
2 plt.plot(means, label='mean')
3 plt.plot(stds, label='std')
4 plt.legend()
5 plt.ylabel('Mutual Information')
6
7 plt.subplot(2, 1, 2)
8 plt.plot(data.mean(axis=1).mean(axis=0))
9 plt.xlabel('time')
10 plt.ylabel('firing rate or Data_mean')

Text(0, 0.5, 'firing rate or Data_mean')

```

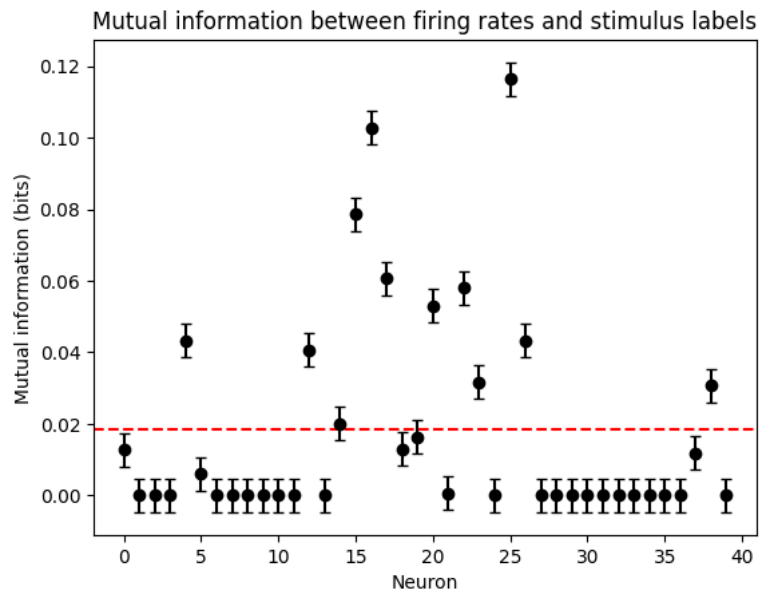


Bar Plot Error

```

1 from sklearn.feature_selection import mutual_info_classif
2
3 # Define the length and stride of the sliding window
4 window_len = 50
5 stride = 1
6
7 # Calculate the number of time bins
8 num_bins = int((data.shape[-1] - window_len) / stride) + 1
9
10 # Initialize an empty array to store the firing rates
11 firing_rates = np.zeros((len(labels), data.shape[1], num_bins))
12
13 # Loop over each trial and each neuron, and calculate the firing rates for each time bin
14 for i in range(len(labels)):
15     for j in range(data.shape[1]):
16         for k in range(num_bins):
17             window_start = k * stride
18             window_end = window_start + window_len
19             num_spikes = np.sum(data[i, j, window_start:window_end])
20             firing_rates[i, j, k] = num_spikes / window_len * 1000 # Convert to firing rate in Hz
21
22 # Calculate the mutual information score for each neuron
23 mi_scores = np.zeros(data.shape[1])
24 for i in range(data.shape[1]):
25     mi_scores[i] = mutual_info_classif(firing_rates[:, i, :], labels)[0]
26
27 # Plot the mutual information scores
28 mean_mi = np.mean(mi_scores)
29 sem_mi = np.std(mi_scores) / np.sqrt(len(mi_scores))
30 plt.errorbar(range(data.shape[1]), mi_scores, yerr=sem_mi, fmt='o', color='black', capsize=3)
31 plt.axhline(y=mean_mi, linestyle='--', color='red')
32 plt.xlabel('Neuron')
33 plt.ylabel('Mutual information (bits)')
34 plt.title('Mutual information between firing rates and stimulus labels')
35 plt.show()
36

```



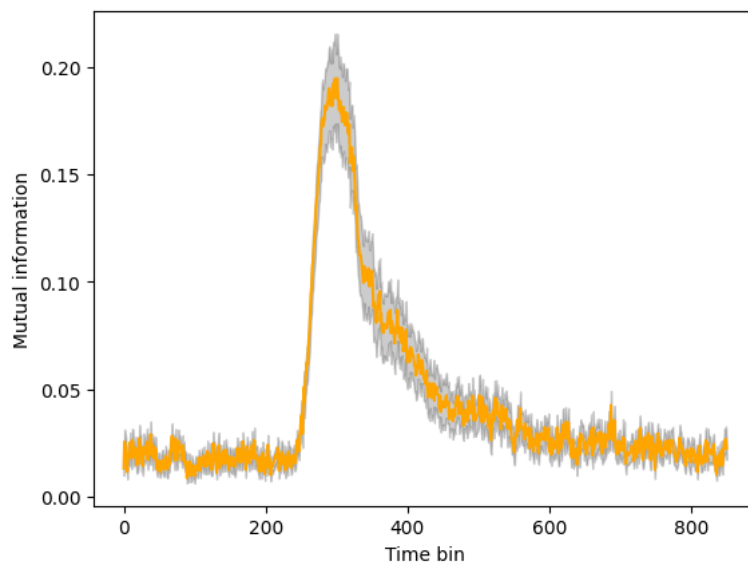
Calculate the mutual information for each time bin

```

1 # Calculate the mutual information for each time bin and each neuron
2 mi_scores = np.zeros((data.shape[1], num_bins))
3 for i in range(data.shape[1]):
4     for j in range(num_bins):
5         mi_scores[i, j] = mutual_info_classif(firing_rates[:, i, j].reshape(-1, 1), labels)[0]
6
7 # Calculate the mean and standard error of the mean of the mutual information across neurons
8 mi_mean = np.mean(mi_scores, axis=0)
9 mi_sem = np.std(mi_scores, axis=0) / np.sqrt(mi_scores.shape[0])

1 # Plot the mean and standard error of the mean of the mutual information
2 fig, ax = plt.subplots()
3 ax.plot(np.arange(num_bins), mi_mean, color='orange')
4 ax.fill_between(np.arange(num_bins), mi_mean - mi_sem, mi_mean + mi_sem, alpha=0.2, color='black')
5 ax.set_xlabel('Time bin')
6 ax.set_ylabel('Mutual information')
7 plt.show()

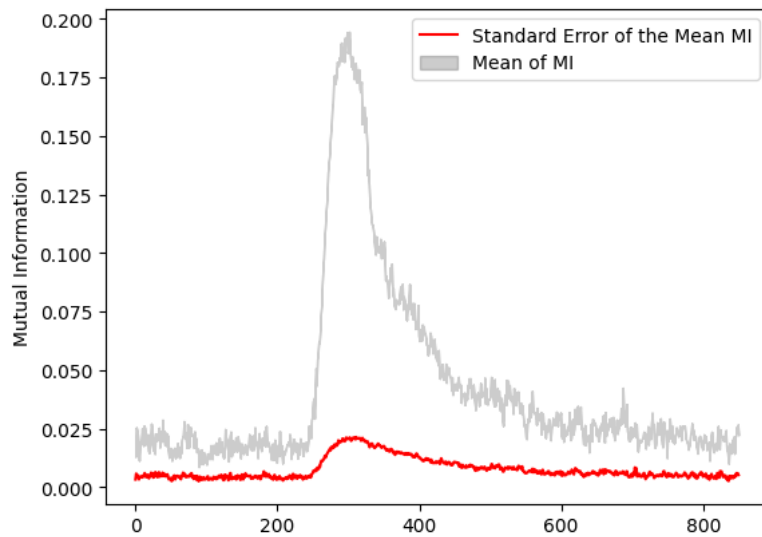
```



```

1 # Plot mean and the standard error of the mean of the mutual information
2 fig, ax = plt.subplots()
3 ax.plot(np.arange(num_bins), mi_sem, color='red', label='Standard Error of the Mean MI')
4 ax.fill_between(np.arange(num_bins), mi_mean - mi_sem, mi_mean + mi_sem, alpha=0.2, color='black', label='Mean of MI')
5 ax.set_xlabel('Time bin')
6 ax.set_ylabel('Mutual Information')
7 ax.legend()
8 plt.show()

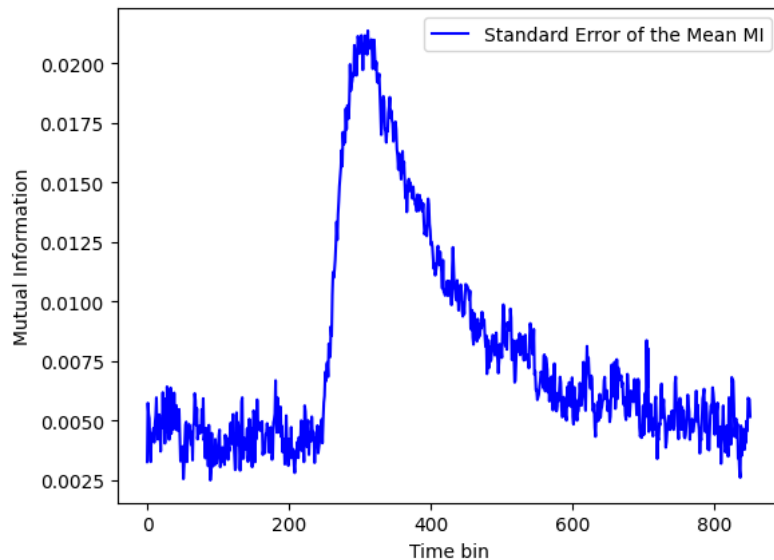
```



```

1 # Plot the standard error of the mean of the mutual information
2 fig, ax = plt.subplots()
3 ax.plot(np.arange(num_bins), mi_sem, color='Blue', label = 'Standard Error of the Mean MI')
4 ax.set_xlabel('Time bin')
5 ax.set_ylabel('Mutual Information')
6 ax.legend()
7 plt.show()

```



Average the d-prime across neurons and plot the results

```

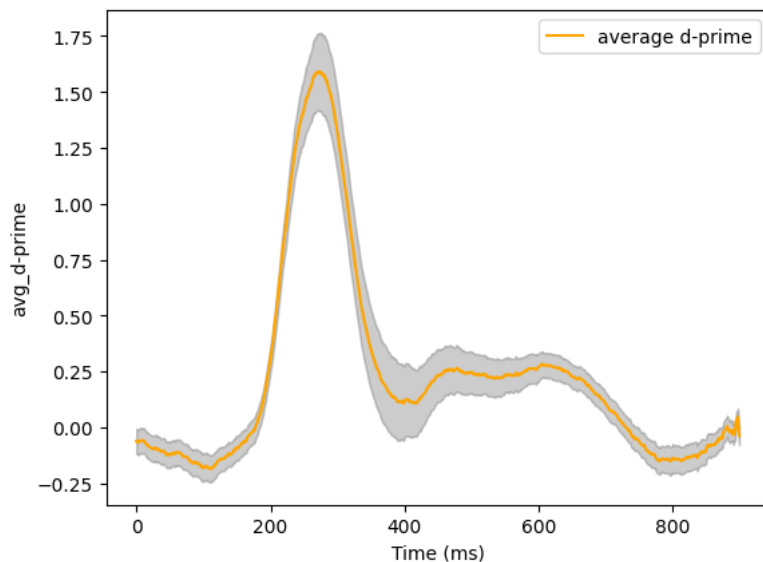
1 # Define window length and compute firing rates for each neuron and stimulus type
2 window_length = 100
3 num_obs, num_neurons, num_samples = data.shape
4 firing_rates = np.zeros((num_obs, num_neurons, num_samples))
5 for i in range(num_obs):
6     for j in range(num_neurons):
7         for k in range(num_samples):
8             window = data[i, j, k:k+window_length]
9             firing_rates[i, j, k] = np.sum(window) / window_length
10
11 # Calculate d-prime over time
12 d_prime = np.zeros((num_neurons, num_samples))
13 labels = labels.reshape((77,))
14 for j in range(num_neurons):
15     for k in range(num_samples):
16         face_firing_rates = firing_rates[labels == 0, j, k]
17         nonface_firing_rates = firing_rates[labels == 2, j, k]
18         face_mean = np.mean(face_firing_rates)
19         nonface_mean = np.mean(nonface_firing_rates)
20         face_var = np.var(face_firing_rates)
21         nonface_var = np.var(nonface_firing_rates)
22         pooled_var = (face_var + nonface_var) / 2
23         epsilon = 1e-6
24         # d_prime[j, k] = (face_mean - nonface_mean) / np.sqrt(pooled_var) + epsilon

```

```

25     if pooled_var < epsilon:
26         d_prime[j, k] = 0 # Set d-prime to 0 when the variance is too small
27     else:
28         d_prime[j, k] = (face_mean - nonface_mean) / np.sqrt(pooled_var) + epsilon
29
30 # Average d-prime across neurons and plot results
31 avg_d_prime = np.mean(d_prime, axis=0)
32 sem_d_prime = np.std(d_prime, axis=0) / np.sqrt(num_neurons)
33 plt.figure()
34 plt.plot(range(num_samples), avg_d_prime, label='average d-prime', color='orange')
35 plt.fill_between(range(num_samples), avg_d_prime - sem_d_prime, avg_d_prime + sem_d_prime, alpha=0.2, color='black')
36 plt.xlabel('Time (ms)')
37 plt.ylabel('avg_d-prime')
38 plt.legend()
39 plt.show()
40

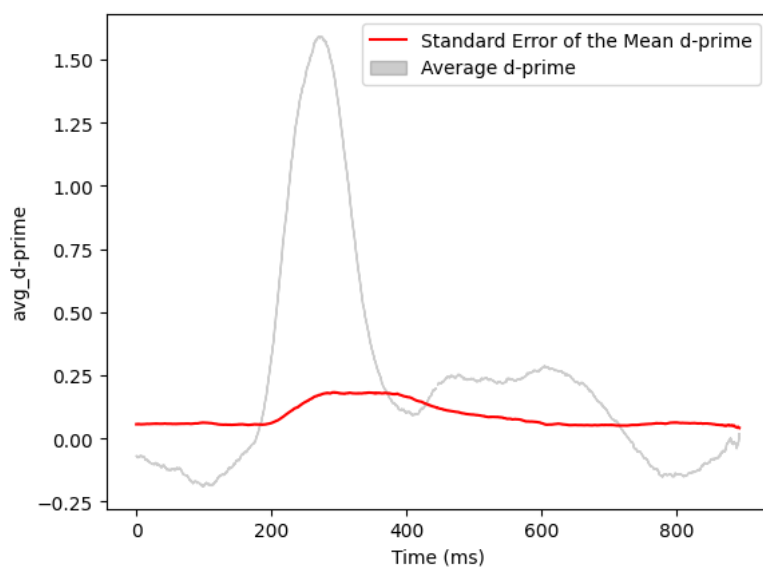
```



```

1 # Plot mean and the standard error of the mean of the d-prime
2 plt.figure()
3 plt.plot(range(num_samples), sem_d_prime, color='red', label='Standard Error of the Mean d-prime')
4 plt.fill_between(range(num_samples), avg_d_prime, avg_d_prime, alpha=0.2, color='black', label='Average d-prime')
5 plt.xlabel('Time (ms)')
6 plt.ylabel('avg_d-prime')
7 plt.legend()
8 plt.show()

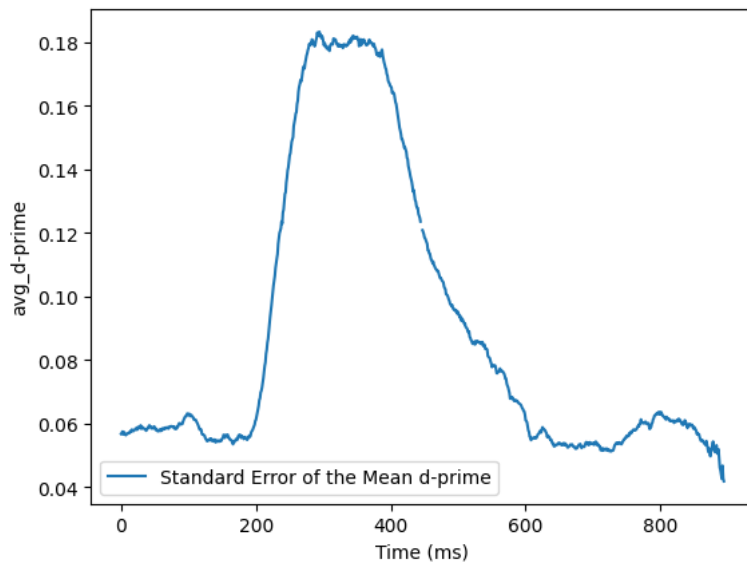
```



```

1 # Plot the standard error of the mean of the d-prime
2 plt.figure()
3 plt.plot(range(num_samples), sem_d_prime, label='Standard Error of the Mean d-prime')
4 plt.xlabel('Time (ms)')
5 plt.ylabel('avg_d-prime')
6 plt.legend()
7 plt.show()

```

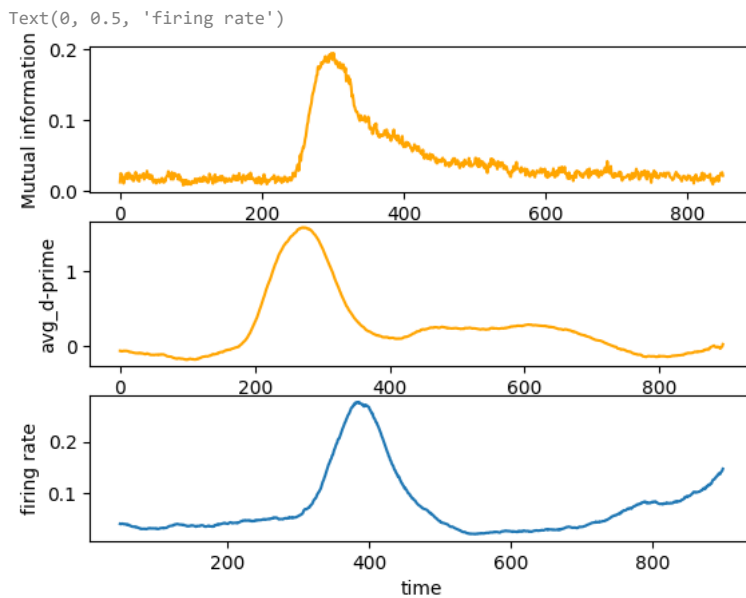



Compare Both Mutual information and average d-prime

```

1 plt.subplot(3, 1, 1)
2 # Plot the mean and standard error of the mean of the mutual information
3 plt.plot(np.arange(num_bins), mi_mean, color='orange')
4 # plt.fill_between(np.arange(num_bins), mi_mean - mi_sem, mi_mean + mi_sem, alpha=0.2, color='black')
5 plt.xlabel('Time bin')
6 plt.ylabel('Mutual information')
7
8 plt.subplot(3, 1, 2)
9 # Average d-prime across neurons and plot results
10 plt.plot(range(num_samples), avg_d_prime, label='average d-prime', color='orange')
11 # plt.fill_between(range(num_samples), avg_d_prime - sem_d_prime, avg_d_prime + sem_d_prime, alpha=0.2, color='black')
12 plt.xlabel('Time (ms)')
13 plt.ylabel('avg_d-prime')
14
15 plt.subplot(3, 1, 3)
16 time = np.arange(window_length//2-1, 900 ) # Adjust x-axis based on window length
17 plt.plot(time, human_face_firing_rate, label='Human Face Stimulus')
18 plt.xlabel('time')
19 plt.ylabel('firing rate')

```



```

1 face_mean = Moving_average[np.where(labels!=2)[0]].mean(axis=0)
2 non_face_mean = Moving_average[np.where(labels==2)[0]].mean(axis=0)
3
4 face_std = Moving_average[np.where(labels!=2)[0]].std(axis=0)
5 non_face_std = Moving_average[np.where(labels==2)[0]].std(axis=0)
6
7 pooled_var = np.sqrt(face_std ** 2 + non_face_std ** 2)/2
8

```

```

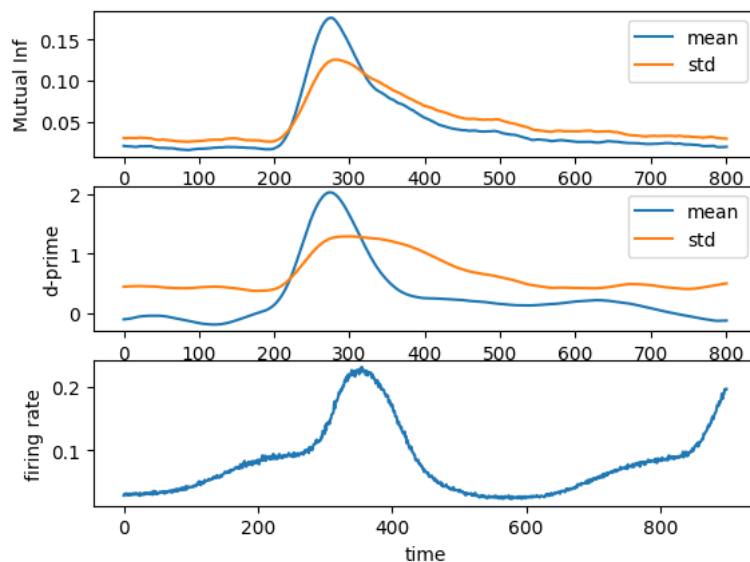
9 epsilon = 1e-6
10 d_prime = (face_mean - non_face_mean) / (pooled_var + epsilon)

1 d_prime_means = pd.DataFrame(np.nanmean(d_prime, axis=0)).rolling(50, axis = 0).mean().dropna(axis=0).to_numpy().reshape(-1)
2 d_prime_std = pd.DataFrame(np.nanstd(d_prime, axis=0)).rolling(50, axis = 0).mean().dropna(axis=0).to_numpy().reshape(-1)

1 plt.subplot(3, 1, 1)
2 plt.plot(means, label='mean')
3 plt.plot(stds, label='std')
4 plt.legend()
5 plt.ylabel('Mutual Inf')
6
7 plt.subplot(3, 1, 2)
8 plt.plot(d_prime_means, label='mean')
9 plt.plot(d_prime_std, label='std')
10 plt.legend()
11 plt.ylabel('d-prime')
12
13 plt.subplot(3, 1, 3)
14 plt.plot(data.mean(axis=1).mean(axis=0))
15 plt.xlabel('time')
16 plt.ylabel('firing rate')
17
18

```

Text(0, 0.5, 'firing rate')



Compare Both the onset and peak timings of the Mutual information and average d-prime

```

1 # Define window length and compute firing rates for each neuron and stimulus type
2 window_length = 100
3 num_obs, num_neurons, num_samples = data.shape
4 firing_rates = np.zeros((num_obs, num_neurons, num_samples))
5 for i in range(num_obs):
6     for j in range(num_neurons):
7         for k in range(num_samples):
8             window = data[i, j, k:k+window_length]
9             firing_rates[i, j, k] = np.sum(window) / window_length
10
11 # Calculate the mutual information for each time bin and each neuron
12 mi_scores = np.zeros((num_neurons, num_samples))
13 for j in range(num_neurons):
14     for k in range(num_samples):
15         mi_scores[j, k] = mutual_info_classif(firing_rates[:, j, k].reshape(-1, 1), labels)[0]
16
17 # Calculate d-prime over time
18 d_prime = np.zeros((num_neurons, num_samples))
19 labels = labels.reshape((77,))
20 for j in range(num_neurons):
21     for k in range(num_samples):
22         face_firing_rates = firing_rates[labels == 0, j, k]
23         nonface_firing_rates = firing_rates[labels == 2, j, k]
24         face_mean = np.mean(face_firing_rates)
25         nonface_mean = np.mean(nonface_firing_rates)
26         face_var = np.var(face_firing_rates)
27         nonface_var = np.var(nonface_firing_rates)
28

```

```

28     pooled_var = (face_var + nonface_var) / 2
29     epsilon = 1e-6
30     # d_prime[j, k] = (face_mean - nonface_mean) / np.sqrt(pooled_var) + epsilon
31     if pooled_var < epsilon:
32         d_prime[j, k] = 0 # Set d-prime to 0 when the variance is too small
33     else:
34         d_prime[j, k] = (face_mean - nonface_mean) / np.sqrt(pooled_var) + epsilon
35 # Calculate the threshold as the mean plus one standard deviation
36 threshold = np.mean(d_prime) + np.std(d_prime)
37
38 # Find the onset and peak timings of d-prime, mutual information, and firing rate
39 onset_dprime = np.argmax(d_prime > threshold)
40 peak_dprime = np.argmax(d_prime)
41 onset_mi = np.argmax(mi_scores > threshold)
42 peak_mi = np.argmax(mi_scores)
43 onset_firing_rate = np.argmax(firing_rates)
44 peak_firing_rate = np.argmax(np.max(firing_rates, axis=2))
45
46 print("Onset timing - d-prime:", onset_dprime)
47 print("Peak timing - d-prime:", peak_dprime)
48 print("Onset timing - mutual information:", onset_mi)
49 print("Peak timing - mutual information:", peak_mi)
50 print("Onset timing - firing rate:", onset_firing_rate)
51 print("Peak timing - firing rate:", peak_firing_rate)
52
53

```

```

Onset timing - d-prime: 201
Peak timing - d-prime: 10179
Onset timing - mutual information: 0
Peak timing - mutual information: 11077
Onset timing - firing rate: 595220
Peak timing - firing rate: 661

```